IG BANK

TRUST IS KEY

TEAM **DEVX**

IG BANK

PROBLEM STATEMENT

Traditional banking operations often rely on tightly coupled, monolithic systems that are difficult to scale, maintain, and monitor.

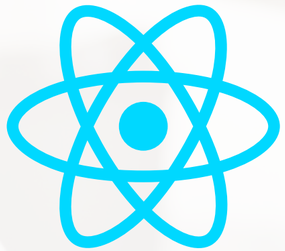
With increasing customer demand for secure online services such as account registration, fund transfers, deposits, withdrawals, and statement generation, banks require a modern, scalable, and highly available system architecture.

This project aims to develop a Banking System using microservices architecture to efficiently support customer and employee banking operations.

The system ensures reliability, and scalability through service discovery and load balancing

IG BANK

TECHNOLOGY USED



FRONTEND

- Built using React with a modern component-based architecture for a responsive and user-friendly UI.
- Communicates with the backend only via the API Gateway using REST APIs.
- Efficient to build Single Page Application(SPA)

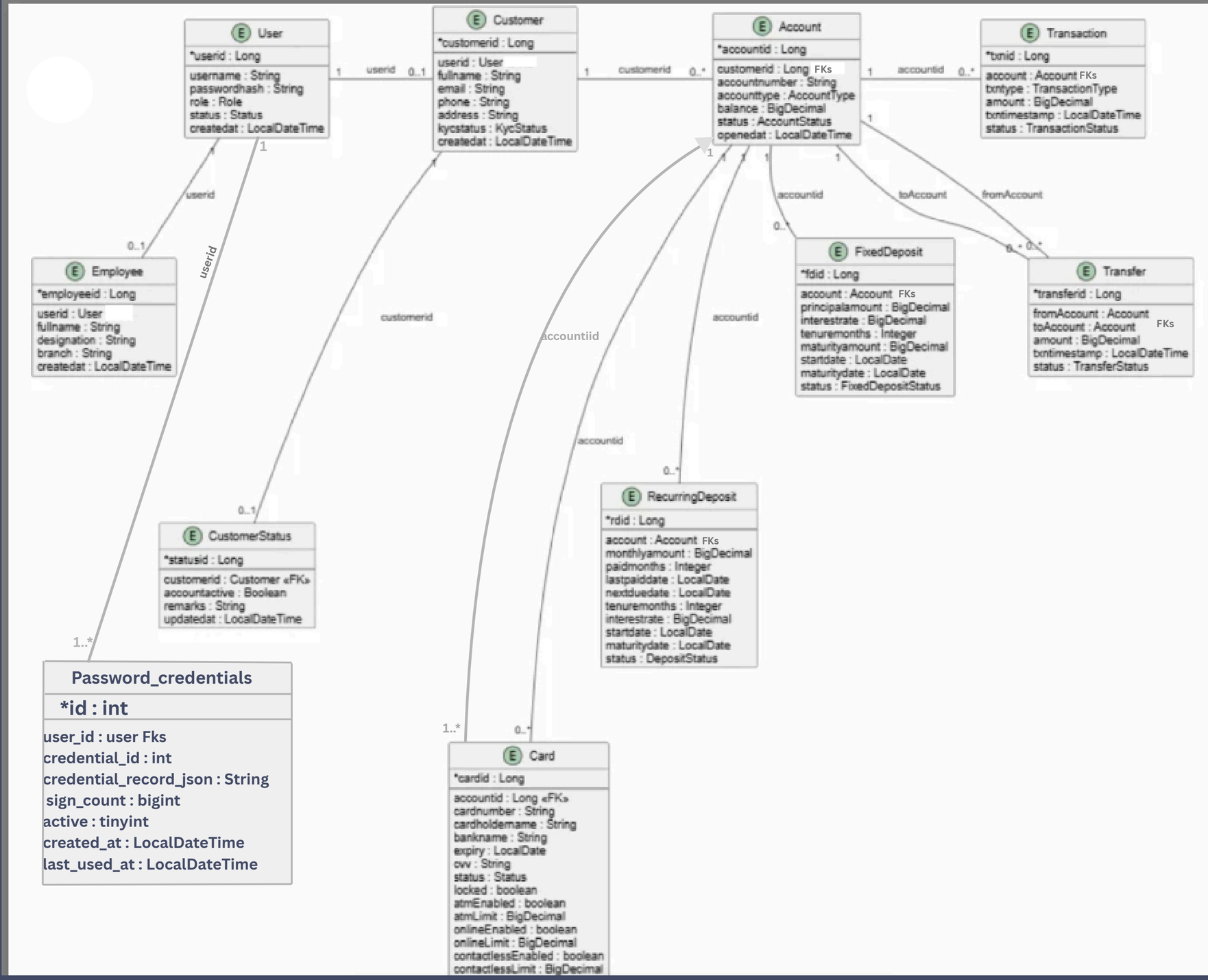
BACKEND

- Supports for REST APIs
- Widely used in Existing Banking Sytem as it is reliable and scalable
- Support Micro Service Architecture
- Spring Cloud is used for Service Discovery

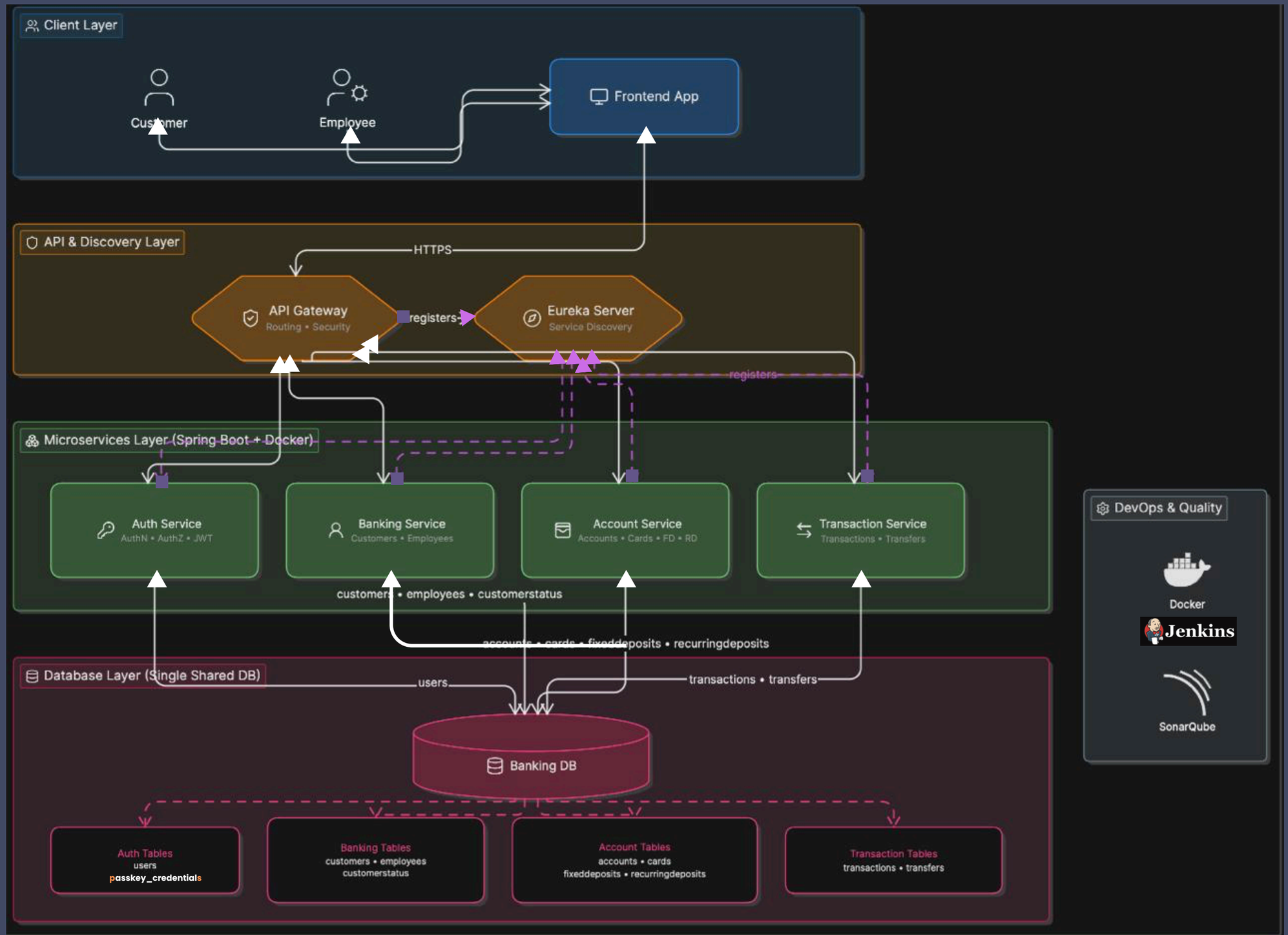
DATABASE

- Implemented using Spring Data JPA for ORM and clean data access.
- MySQL is used as the relational database to store transactional and user data.
- Ensures ACID properties, which is critical for financial systems to maintain data integrity and prevent inconsistencies.

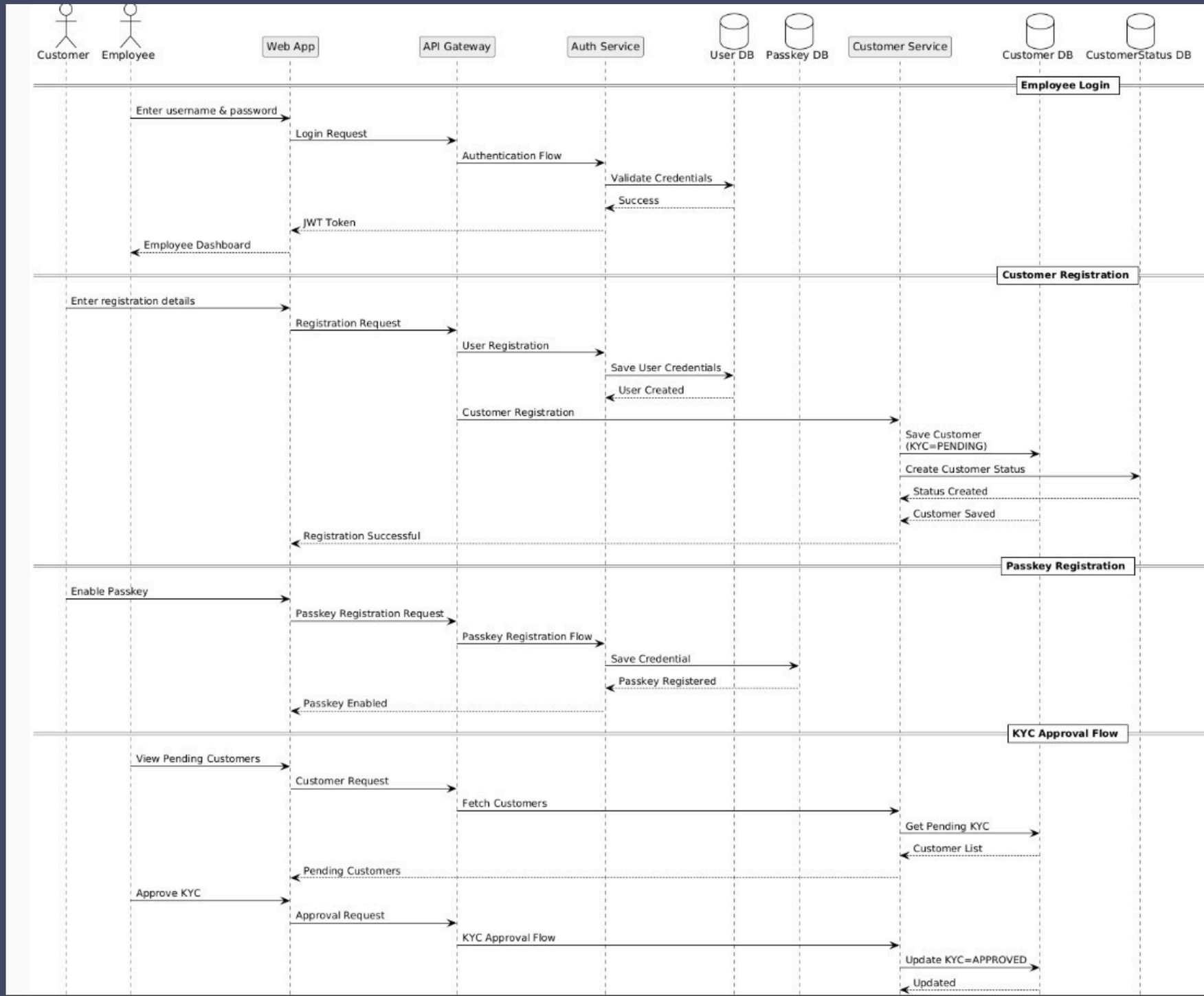
SCHEMA DIAGRAM



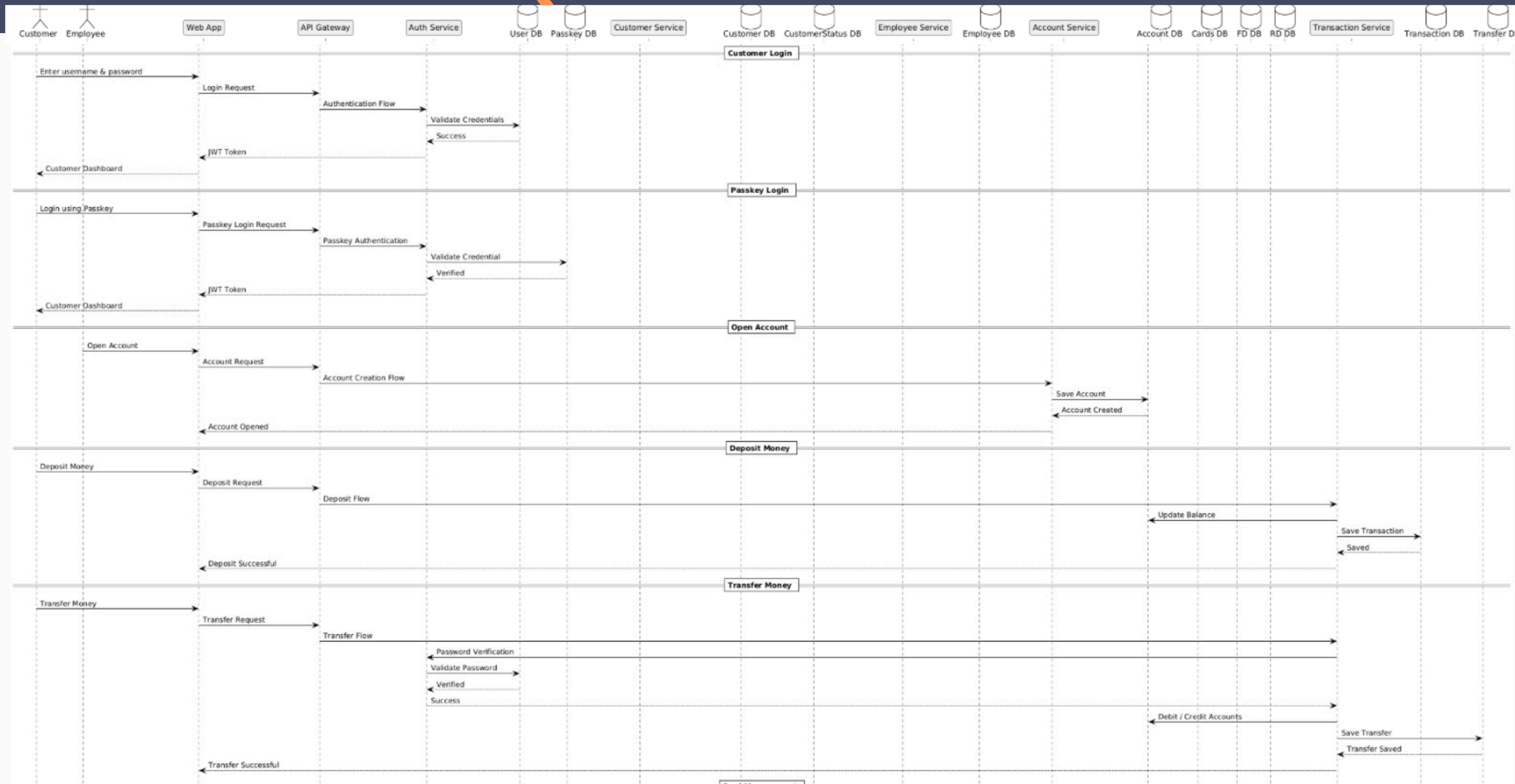
HIGH LEVEL ARCHITECTURE



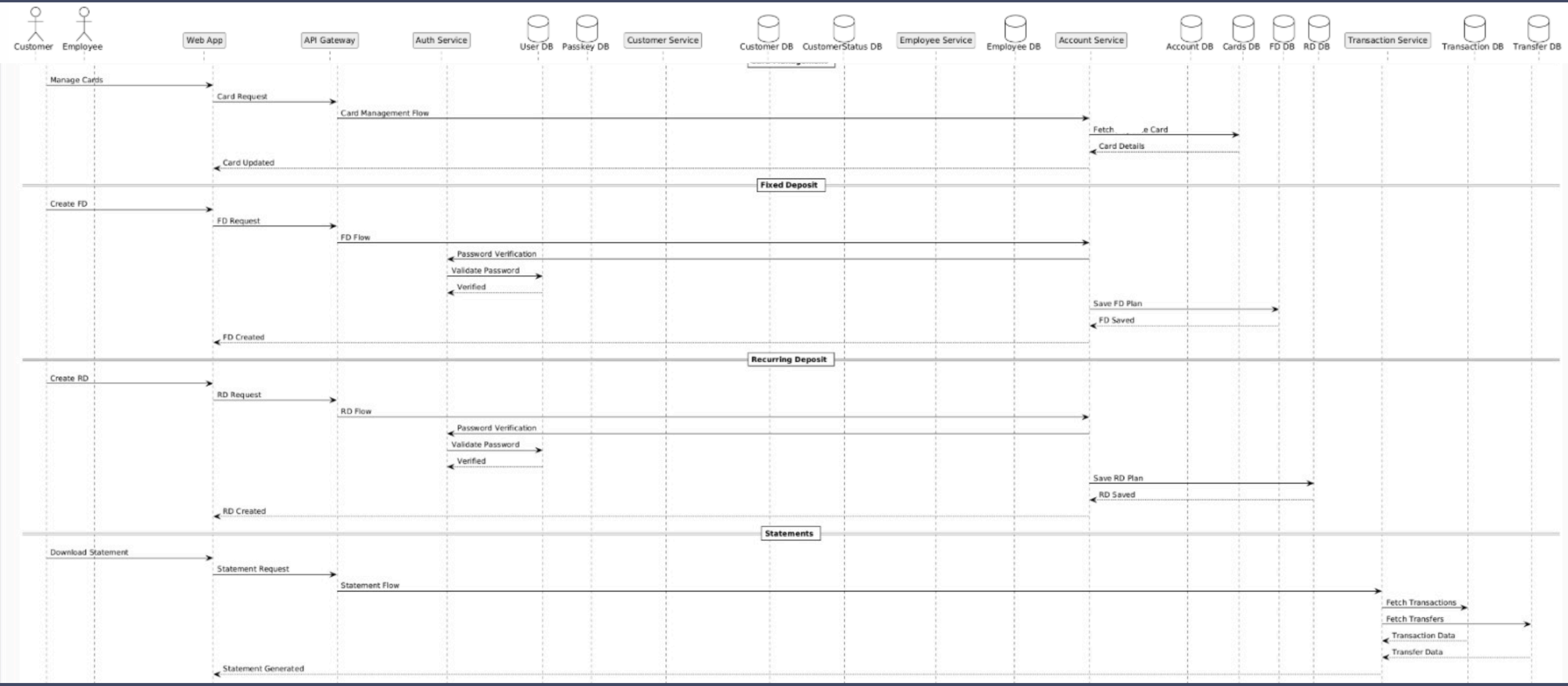
SEQUENCE DIAGRAM



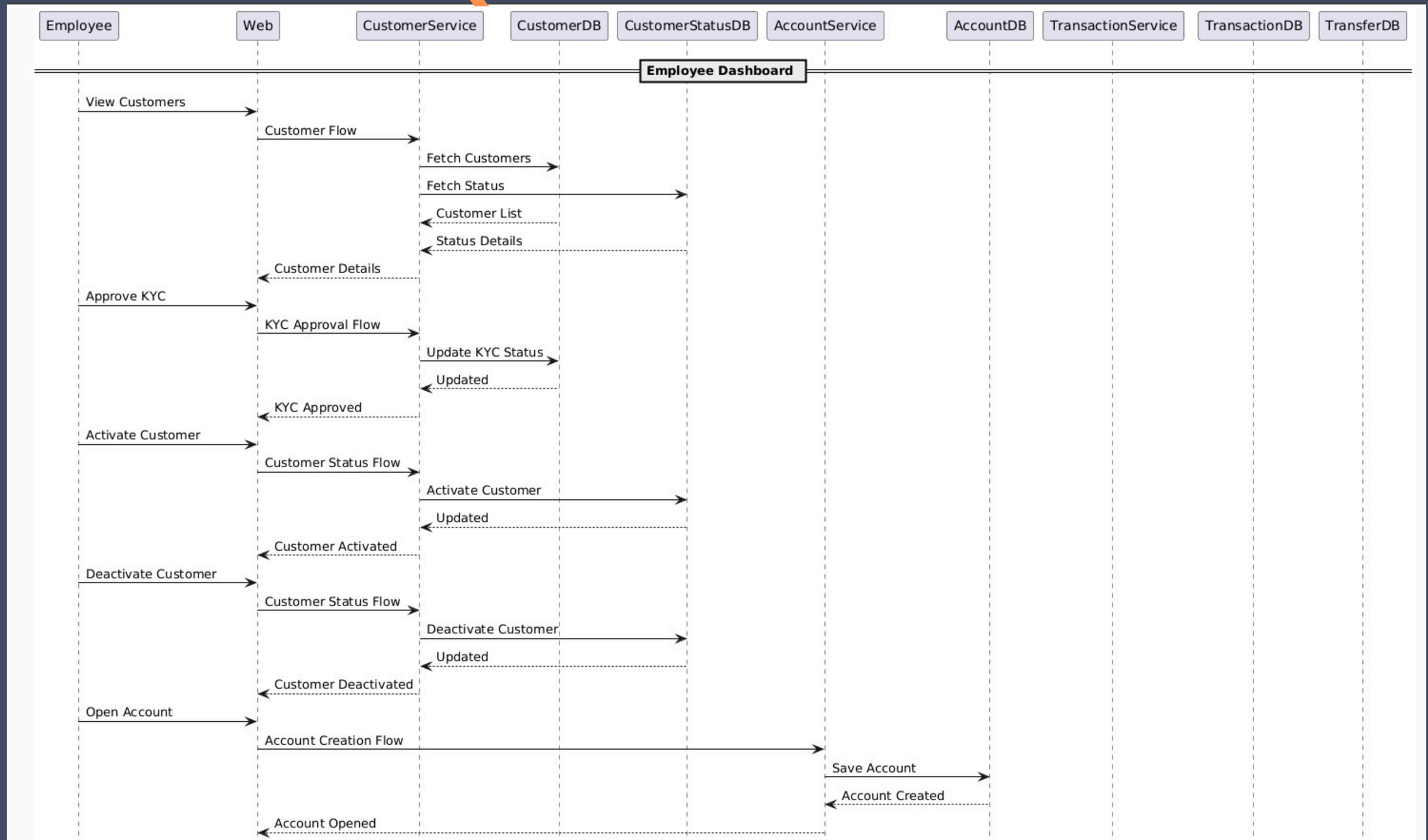
SEQUENCE DIAGRAM



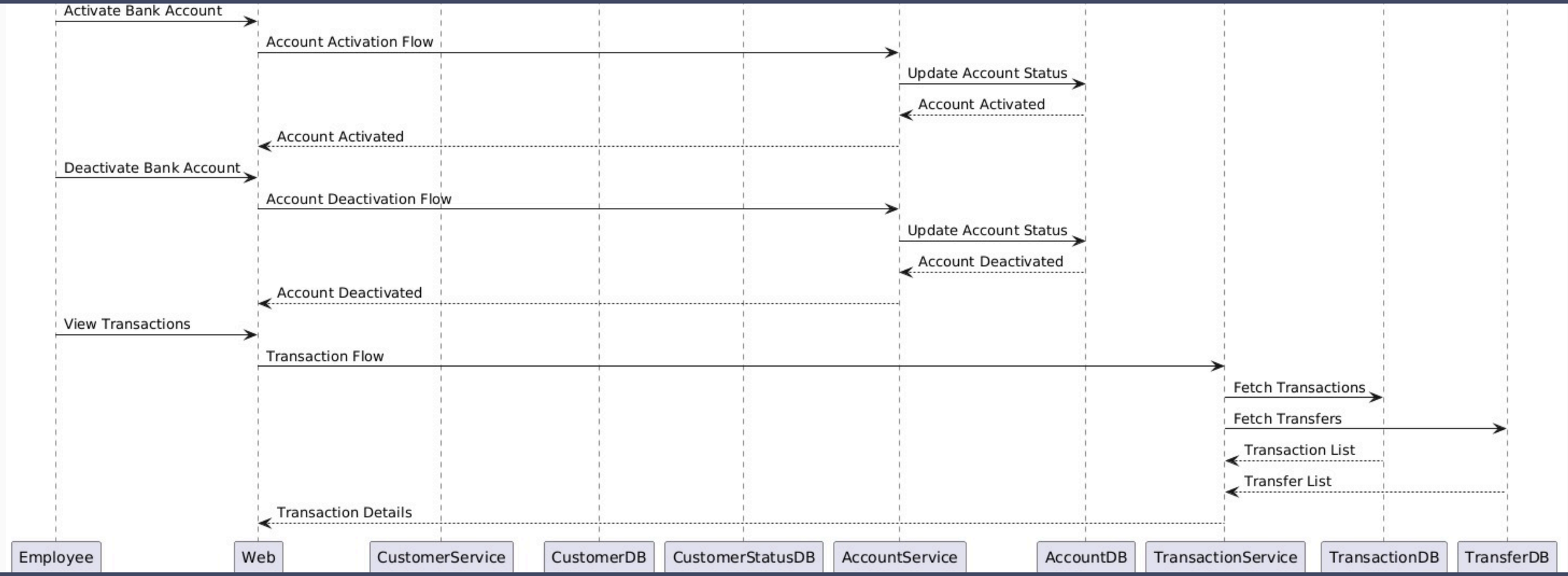
SEQUENCE DIAGRAM



SEQUENCE DIAGRAM



SEQUENCE DIAGRAM



SERVICES

Eureka Server

- Instead of every service manually knowing the IP and port of every other service, they all register themselves with the Eureka Server.
- Eureka Server acts as a service registry
- In our application, we have enabled Eureka using the @EnableEurekaServer annotation.
- The Application runs on port 8761, which is the standard port for Eureka Server.
- All other microservices connect to it using the configured defaultZone URL.
- Eureka Server improves system reliability by continuously monitoring services and removing inactive instances automatically.

SERVICES

API Gateway

- What is API Gateway ?
  Single entry point for all client requests in microservices
  Acts as a reverse proxy between clients and backend services

- **Why API Gateway?**
- Clients do not need to be aware of all individual microservices
- Eliminates the need to implement security separately in every service
- Reduces complexity in client-side logic

- Role in Our System (Important)
  Client → API Gateway → Auth /Customer /Account /Transaction Services
  Validates JWT token before reaching services
  Sends only trusted, authenticated requests to backend

SERVICES

Load Balancer in API Gateway

- Distributes incoming requests across multiple service instances
- Ensures high availability and fault tolerance
- Improves performance and scalability
- Enables dynamic scaling without client-side changes
- Works with service discovery to route requests intelligently

SSL IMPLEMENTATION

What is SSL?

- Protocol that secures communication using encryption (HTTPS)

How it Works:

- Client requests connection → Server sends certificate → Client verifies → Secure encrypted session starts

Implementation Steps:

- Obtain SSL certificate
- Install on server
- Enable HTTPS
- Redirect HTTP → HTTPS
- Test configuration

Key Features:

- Encryption (data protection)
- Authentication (verify server identity)
- Data integrity

Benefits:

- Protects sensitive data
- Prevents cyber attacks

SERVICES

Auth Service

End-to-End Flow

- User login → credentials validated → JWT issued
- Client sends JWT in every request
- JWT Filter intercepts request
- Token validated (signature + expiry)
- User + role extracted → stored in SecurityContext
- Authorization rules applied → request allowed/denied
- Server does NOT store session
- JWT carries everything

Risk & Best Practices

- Secret keys must never be exposed/logged
- Use strong signing keys for security
- Token expiry ensures time-bound access control

SERVICES

AUTH Service

Step-by-Step Flow

1. **User requests OTP**
   - User enters email and clicks “Send OTP”
2. **Auth Service processes request**
   - Generates OTP
   - Stores OTP temporarily (with expiry time)
   - Prepares email content (OTP message)
3. **Mail layer is triggered**
   - Spring Boot uses JavaMailSender
   - Converts email into SMTP request
4. **SMTP server interaction**
   - Authenticates using email credentials
   - Accepts the email request
5. **Email routing**
   - SMTP server sends email over internet
   - Finds recipient mail server
6. **Recipient mail server**
   - Receives email
   - Stores it in user mailbox
7. **User receives OTP**
   - OTP delivered to user inbox
   - User uses OTP for verification

SERVICES

AUTH Service

What is a Passkey?

- A passwordless login method based on public-key cryptography
- Uses a private key (device) and public key (server)
- Server never stores or sees the private key

Registration (Sign-Up) Flow

1. User logs in normally
2. Server generates a unique challenge
3. Browser creates a key pair
4. Private key stays on device
5. Public key is sent and stored on server

Authentication (Login) Flow

1. User enters username
2. Server sends new challenge
3. User verifies via fingerprint/face/PIN
4. Device signs challenge using private key
5. Server verifies using stored public key
6. User gets logged in (JWT/session created)

SERVICES

Account Service

- Customer can create a new bank account
- Customer can retrieve account details
- Uses proper HTTP methods (GET, POST) following REST standards
- Supports account lifecycle management through dedicated activate and deactivate endpoints
- Account Service runs on port 8901 but is accessed via a centralized API Gateway

SERVICES

Account Service

- Employee can open accounts for customers
  - Ensures:
    - KYC validation
    - Proper account setup

- Employees can:
  - Activate accounts after verification
  - Deactivate suspicious or inactive accounts
- Account Service runs on port 8901 but is accessed via a centralized API Gateway

SERVICES

Banking System(User) Service

- Provides REST APIs for customer and employee registration, retrieval, update, activate and inactivate.
- Supports multiple lookup options using ID, userId, email, and phone number
- Manages customer lifecycle and verification, including activation, deactivation, and KYC handling
- Enables employee-specific operations such as customer management and validation checks
- Core Banking System service runs on port 8900 and is accessed through a unified API Gateway

SERVICES

Transaction Service

- The Transaction Service is responsible for core banking operations like deposit, withdrawal, and transfer, ensuring money moves correctly between accounts.
- All transactions are recorded, so the system always has a complete audit trail for accounts.
- It checks conditions such as account active status, sufficient balance, and positive transaction amounts before processing any operation.
- It generates a user-friendly transaction statement showing credits, debits, timestamps, status, and references for UI display.
- The service cleanly separates controller (API layer), service (business logic), and repository (database access) for maintainability and clarity.

SUPPORT

CHATBOT

- **ML-Based Chatbot (Frontend)**
  Understands user queries (transfer, deposit, statement) using lightweight machine learning.
- **Secure & Privacy-Focused**
  Blocks sensitive data access (balance, account details, OTP/PIN) with security-aware responses.
- **Guided Assistance & Navigation**
  Provides step-by-step instructions with quick access buttons to key features.
- **Stateless Session Design**
  No data storage—ensures privacy and does not retain previous conversations.

SUPPORT

STATEMENT ANALYSIS

- **Pipedream Integration**

  Acts as a middleware between backend and external services

  Simplifies API integration without complex authentication

  Used to:

  Upload reports to Google Drive

  Connect with AI services

- **Google Drive Integration**

  Backend sends report to Pipedream via webhook

  Pipedream uploads PDF directly to user’s Google Drive

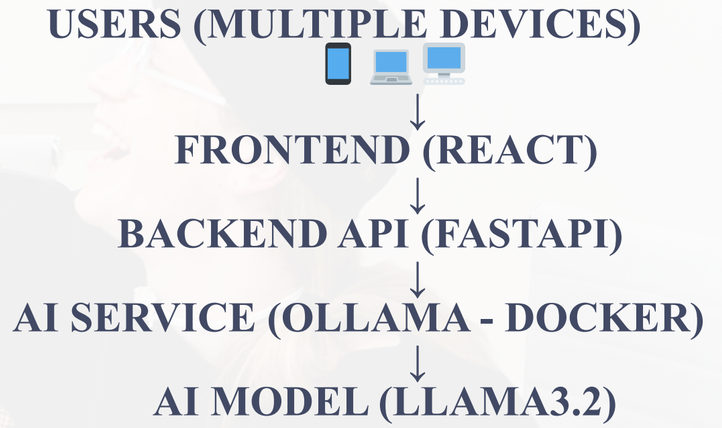
SUPPORT

STATEMENT ANALYSIS

- **LLM Integration (Ollama + Docker)**
- Integrated a local LLM using Ollama to generate AI-driven financial insights
- Deployed the AI model inside a Docker container, ensuring consistent and portable execution
- Enables scalable and isolated AI service as a microservice within the system

◆ **Key Benefit**

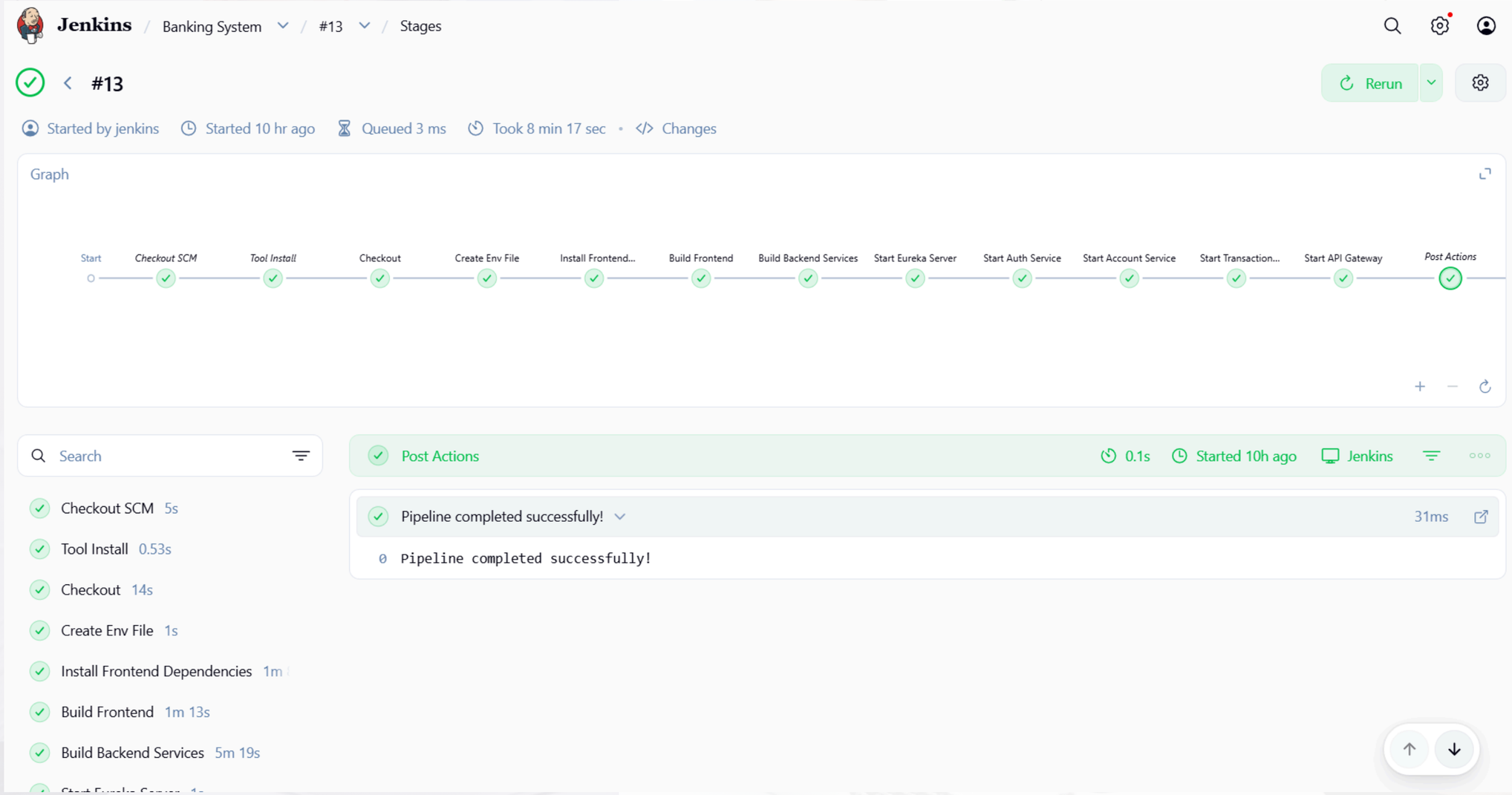
- Combines **Data Analytics** + **Automation** + **AI** to provide smart financial recommendations



JENKINS

Jenkins / Banking System / Full Stage View
Banking System - Stage View
Declarative: Checkout SCM
Declarative: Tool Install
Checkout
Create Env File
Install Frontend Dependencies
Build Frontend
Build Backend Services
Start Eureka Server
Start Auth Service
Start Account Service
Start Transaction Service
Start API Gateway
Declarative: Post Actions
Average stage times:
(full run time: ~8min 17s)
4s 665ms 9s 1s 1min 8s 1min 13s 4min 1s 615ms 573ms 546ms 520ms 500ms 195ms
#13 May 26 00:31 1 commit
5s 606ms 14s 1s 1min 8s 1min 13s 5min 19s 1s 1s 1s 1s 1s 157ms
#12 May 26 00:18 1 commit
5s 704ms 8s 1s 1min 8s 1min 12s 2min 21s failed 161ms failed 163ms failed 167ms failed 168ms failed 176ms failed 183ms
#11 May 26 00:04 3 commits
4s 686ms 4s 1s 1min 9s 1min 13s 4min 22s failed 214ms failed 199ms failed 193ms failed 202ms failed 191ms failed 246ms
#10 May 25 23:59 No Changes
#9 May 25 23:56 No Changes
#8 May 25 23:55 No Changes

JENKINS


Jenkins / Banking System / #13 / Stages
#13
Rerun
Started by jenkins
Started 10 hr ago
Queued 3 ms
Took 8 min 17 sec
Changes
Graph
Start
Checkout SCM
Tool Install
Checkout
Create Env File
Install Frontend...
Build Frontend
Build Backend Services
Start Eureka Server
Start Auth Service
Start Account Service
Start Transaction...
Start API Gateway
Post Actions
Search
Checkout SCM 5s
Tool Install 0.53s
Checkout 14s
Create Env File 1s
Install Frontend Dependencies 1m
Build Frontend 1m 13s
Build Backend Services 5m 19s
Post Actions
0.1s
Started 10h ago
Jenkins
Pipeline completed successfully!
31ms
0 Pipeline completed successfully!

TESTING

Quality Gate

Passed

Last analysis 6 hours ago

The last analysis has warnings. See details

New Code | Overall Code

New Code **Since March 24, 2026** **Started 1 month ago**

New issues

0

Required = 0

Accepted issues

0

Valid issues that were not fixed

Coverage

81.4%

Required ≥ 80.0%

On **584** New Lines to cover.

TESTING

Quality Gate

Passed

Last analysis 35 minutes ago

The last analysis has warnings. See details

New Code | Overall Code

New Code **Since March 24, 2026** **Started 1 month ago**

New issues

0

Required = 0

Accepted issues

0

Valid issues that were not fixed

Coverage

82.7%

Required ≥ 80.0%

On **533** New Lines to cover.

TESTING

Quality Gate

Passed

Last analysis 6 hours ago

The last analysis has warnings. See details

New Code | Overall Code

New Code **Since March 24, 2026** **Started 1 month ago**

New issues

0

Required = 0

Accepted issues

0

Valid issues that were not fixed

Coverage

81.4%

Required ≥ 80.0%

On **584** New Lines to cover.

IG BANK

Challenges Faced

- **Merge Conflict Error**

“Encountered Git merge conflicts during parallel development, which were resolved by carefully reviewing code differences and systematically merging changes to maintain code consistency.”

- **CORS errors during frontend–backend integration**

“We faced Cross-Origin Resource Sharing (CORS) errors because the React frontend and backend services ran on different ports, and browsers block such requests by default for security reasons. We solved this by configuring CORS at the API Gateway and backend level, explicitly allowing frontend origins, HTTP methods, and headers. This ensured secure communication without exposing the services.”

- **External Service Integration Challenges**

“Faced challenges with direct API authentication and integration complexity, which was overcome by using Pipedream as a middleware to simplify secure communication and workflow automation.”

- **OTP Generation Challange**

“Standard authentication methods failed to send OTP emails reliably due to security restrictions on external email services .Resolved by configuring app passwords for secure email authentication, enabling successful OTP delivery and verification.”

IG BANK

FUTURE ENHANCEMENTS

BILLS

- Enables users to pay utility bills, mobile recharges, and subscriptions from one platform.
- Provides reminders and history tracking for recurring bill payments.

PAYMENT GATEWAY

- Secure integration for processing online transactions in real time.
- Supports multiple payment modes with encryption and fraud protection.

ANALYSIS ENHANCEMENT

- Integrate with Eleven Labs for AI call assisstant to provide 24X7 support
- Reduces Man Power and Maintainanace Cost

NOTIFICATIONS

- Sends real-time alerts for transactions, bill due dates, and account activities.
- Improves user awareness and enhances security through instant updates.

IG BANK

SPRINT PLANNING

Sprint 1

Task	Storyline	Story Pts.
Requirement Analysis	Gathering and documenting all functional and non-functional requirements and banking flow and scope	3
Sprint Planning	Planning sprint workflow, timelines, feature allocation, and implementation strategy.	5
System Design	Designing database schema, ER diagrams, API structures, frontend architecture, and component diagrams.	8
Backend Setup	Setting up Spring Boot microservices architecture, Eureka Server, and API Gateway.	5
Frontend Setup	Designing frontend structure including Home, Login, Signup, and Dashboard pages.	3
User Service	CRUD For Customer and Employee and managing with accounts	3
Account Service	Account creation and balance logic implementations, validations for the accounts	5
Testing	Performed unit testing to test CRUD operations and basic logic checking	3

Total: 35

Sprint 2

Task	Storyline	Story Pts.
Authentication Service	Implementing secure Register and Login services using JWT authentication and role-based authorization.	5
Transaction Service	Creating services to manage banking operations (deposit, withdraw and transfer validations)	5
Frontend Development	UI and API Integration, Handling all the states	5
Integration of API Gateway and Eureka	Service communication setup as a unified flow	5
Chatbot Module	Developing chatbot support system for user guidance, and user assistance.	5
Frontend Integration	Integrating React frontend components with backend APIs and ensuring smooth data flow.	8
Security	Implemented SSL and Passkey login.	3
Testing & Validation	Performing API testing, frontend validation, integration testing, bug fixing and deployment through Jenkins pipeline.	5

Total: 38

IG BANK

THANK YOU